ENEE 467: Robotics Project Laboratory

Putting it All Together: Autonomous Manipulation on the UR3e-HandE

Lab 7

Learning Objectives

The purpose of this lab is to provide a hands-on introduction to 3D perception, motion planning, and control on physical hardware. Students will work with point-cloud–based pose estimation using ROS 2 and the Intel RealSense D435i RGB-D camera, and perform motion planning and control on the UR3e robot using MoveIt 2 and `ros2_control`. After completing this lab, students should be able to:

- Integrate point cloud-based visual perception with motion planning
- Plan and execute manipulation trajectories based on visual feedback
- Understand the perception-to-action pipeline in ROS 2
- Demonstrate an autonomous pick-and-place routine on the real UR3e-Hand-E robot
- Compare the real execution with the simulation to understand sim2real differences

Autonomous robotic manipulation unites perception, planning, and control to enable robots sense, reason, and act independently toward the realization of a prescribed task. Among these elements, this lab focuses on perception and control to enable the real UR3e robot to detect and plan motions to an object in its workspace from point cloud input. Through this lab, students will explore how these subsystems interact in real time, in simulation and partly in hardware.

Lab Instructions

Follow the lab procedure closely. Some code blocks have been populated for you and some you will have to fill in yourself. Pay close attention to the lab Questions (highlighted with a box). Answer these as you proceed through the lab and include your answers in your lab writeup. There are several lab Exercises at the end of the lab manual. Complete the exercises during the lab and include your solutions in your lab writeup.

Autonomous Object Relocation

While conceptually simple, autonomous *object relocation*, or more commonly, pick-and-place (PnP), integrates all aspects of robotic manipulation, viz:

1. **Perception**: Detect and estimate the pose of the object in 3D space.
2. **Motion Planning**: Generate a collision-free trajectory to reach the object, execute the grasp, and move to the goal pose.
3. **Execution**: Command the manipulator to follow the planned motions and monitor execution

for success or failure.

List of Questions

List of Exercises

Hardware Safety

Throughout this manual, boxes beginning with a **Question #** label contain questions you must answer as you progress through the lab:

> **Question 0.** A question for you to consider as you work.

⚠ **Safety and Setup Notices.** Warnings highlight critical steps for hardware safety, teach-pendant operation, and common setup pitfalls. Always read these carefully before commanding motion on the UR3e.

> ⚠ Keep clear of the robot workspace when running autonomous motion. Confirm that the pendant is in `Play` mode before proceeding.

Notes. These boxes provide background or theoretical context for the UR robot system. They are optional during lab time but useful for your report:

> UR3e kinematics are defined in a six-joint chain; MoveIt uses the URDF and joint limits to compute feasible IK solutions.

Code, commands, and output are formatted as follows:

```
ros2 launch ur_robot_driver ur_control.launch.py ...
```

```
[INFO] [launch]: UR3e driver successfully connected.
```

Files and directories appear in magenta teletype font (e.g., `~/lab7_ws/src`), while in-text code and variables use blue teletype (e.g., `object_pose`). Python type hints or XML tags appear in green (e.g., `str`, `<param>`).

1. Test the Autonomous Manipulation Pipeline in Simulation

Before touching hardware, you will complete and verify the pick-and-place plan in simulation. You will fill in the inverse-kinematics step of the autonomous routine, watch the full pick-and-place run safely in Gazebo, and record baseline metrics. Completing and checking the plan here means you never debug it on the real arm: the same inverse-kinematics logic you verify now is what you deploy on hardware later in this lab. Confirm that you have the simulator Docker image by running the following command:

```
docker image ls | grep lab-7-sim
```

Confirm that `lab-7-sim-image` is printed to the terminal. Then run the following command:

```
cd ~/Labs/lab-7-sim/docker
```

Next, start the **pre-installed** simulator Docker container using:

```
userid=$(id -u) groupid=$(id -g) docker compose -f lab-7-sim-compose.yml \
run --rm lab-7-sim-docker
```

> The simulator image is pre-installed on the lab machines, and the repository's `src` directory is mounted into the container read-write. Edits you make to the source on the host (or through the attached VS Code window) take effect inside the container after a rebuild.

1. In the Docker container, source your ROS 2 workspace:

   ```
   cd ~/ros2_ws
   colcon build --symlink-install
   source install/setup.bash
   ```

2. Launch the simulated UR3e–Hand-E robot with the perception, planning, and control nodes:

   ```
   ros2 launch ur3e_hande_bringup sim.launch.py
   ```

3. Verify that separate Gazebo and RViz windows appear displaying the following:
   - The UR3e–Hand-E robot model loaded in Gazebo.
   - Segmented point cloud and plane RViz markers for the table and object (red cylinder). Note that the red cylinder is a proxy for the "cup" in the real environment. Simulating grasping in Gazebo has been a longstanding issue.
   - The point cloud based object pose action server should also display a prompt indicating successful detection and publishing of several objects
4. The pick-and-place node `pnp_demo_sim.py` (in the `ur3e_hande_moveit_py` package) drives the autonomous routine, but its inverse-kinematics step has been left for you to complete. You will fill it in, run the node, and verify the full pick-and-place in Exercise 1. When the routine runs correctly, the node prints a per-subtask report ending with the lines below, and the robot moves through the phases in Figure 1.

   ```
   Execution completed: SUCCEEDED
   Retreated to home; closing gripper.
   Closing gripper...
   Pick-and-place sequence complete.
   ```

Question 1. Which ROS 2 nodes launch during the simulation test above? Enumerate these nodes, and classify the nodes in your list into **Perception**, **Planning**, and **Control** using a table. (**Hint**: Run: `ros2 node list`, while the simulation is active. The node names are descriptive enough to identify their respective classes. Exclude RViz nodes.)

Exercise 1. Verify the Pick-and-Place Plan in Simulation (35 pts.)

The autonomous routine in `pnp_demo_sim.py` computes a joint configuration for each sub-task (approach, grasp, lift, place) with inverse kinematics, then plans and executes to it. The inverse-kinematics step has been left for you to complete. You will finish it, verify that the whole pick-and-place runs in simulation, and record baseline metrics. The same approach is what you deploy on the real robot in the final exercise, so getting it right here means you do not debug it on hardware.

Deliverables:

Complete and verify the plan:

a. Open `pnp_demo_sim.py` and find the `TODO (Sim IK)` block in the `start_sequence` method. Replace each `None` with the matching `self._ik_rtb(...)` call. The target position and IK seed are named in each comment. Rebuild the package, then run the node:

```
ros2 run ur3e_hande_moveit_py pnp_demo_sim
```

Verify in Gazebo that the arm completes the full sequence (approach, grasp, lift, place). Submit your completed inverse-kinematics lines and a short video OR a collage of screenshots showing object detection and plane segmentation in RViz and the robot at the approach and place configurations (see Figure 1).

Planning metrics (baseline for hardware):

b. Terminate all running ROS 2 processes, then relaunch the simulator with metrics enabled (this restarts your completed pick-and-place node automatically):

```
ros2 launch ur3e_hande_bringup sim.launch.py pnp:=true \
print_metrics:=true max_vel_scale:=0.25
```

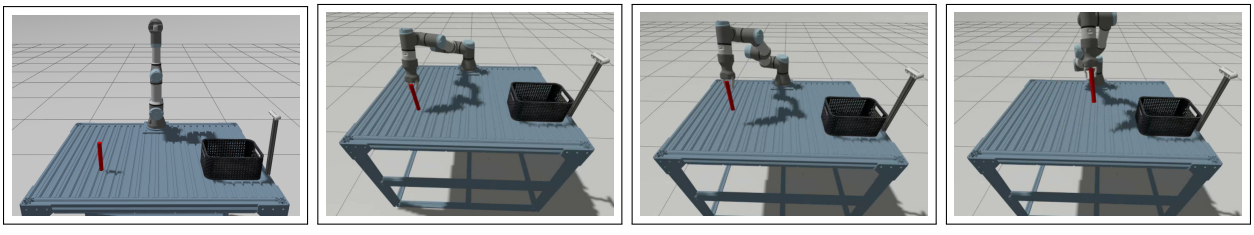
where `max_vel_scale` scales the velocity of each arm joint. Once the task completes, the following metrics are printed to the shell (your numbers may differ, but the task should succeed):

```
-----------------------------------------------------
Metrics Summary:
   1. Planning Success (bool): 1
   2. Planning time [s]: 0.0085 s
   3. Execution time [s]: 4.0520 s
   4. Time (PnP Pipeline) [s]: 34.3 s
-----------------------------------------------------
```

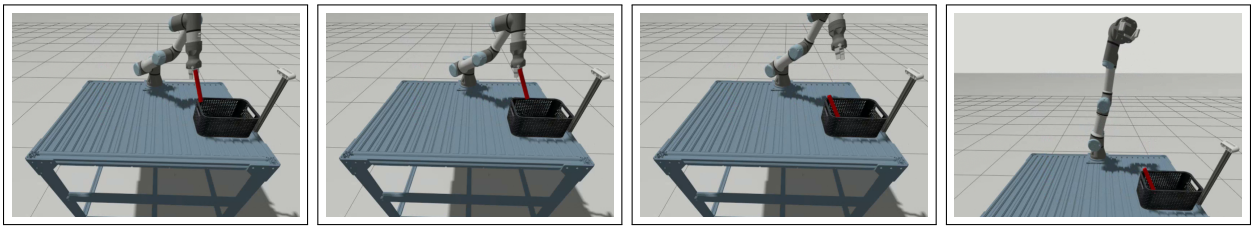
Record the printed metrics in your report.

c. Repeat the metrics run for `max_vel_scale` of 0.20, 0.15, and 0.1. Copy the metrics into

your report. These are the baseline you will compare against on hardware (parts d. and e.).



(a) Task Init (Home) **(b)** Approach/Pre-Grasp **(c)** Grasp/Grip, Lift **(d)** Move to Place



(e) Move to Place **(f)** Place **(g)** Open Gripper **(h)** Retreat to Home

Figure 1: Phases of the object relocation task visualized in simulation.

After completing the steps above (except d. and e.), proceed with the rest of the lab procedure.

Sim2Real caveat. The simulation you just ran is an idealization of the hardware. The perception, planning, and control nodes are the same, but the real system differs in ways that affect performance. The camera extrinsics and intrinsics must be calibrated, depth measurements are noisy and incomplete near object edges, and contact and grasping physics are only approximated in simulation (see the Gazebo grasping note above). Expect to retune thresholds, such as segmentation distances and approach offsets, in moving from simulation to hardware, and treat the simulated metrics as a baseline rather than a prediction of real performance.

STOP: SWITCH CONTAINERS HERE. Everything above ran in the **simulation** container (built from `lab-7-sim-image`). Everything from this point on runs in the **hardware** container (built from `lab-7-hw-image`). Exit the simulation container now, and run every command in the remainder of this lab from a container started from `lab-7-hw-image`, as set up in the next subsection.

2. Hardware Lab Procedure

> ⚠ Stay alert when working with the robot, and use the Emergency Stop Button on the Teach Pendant (see Figure 2a) if needed. Always inform the team member(s) at the hardware station before sending commands to the robot.

2.1 Prerequisites

This lab assumes familiarity with ROS 2, MoveIt, and Universal Robotics's (UR) robot drivers and hardware interface. Prior experience with the teach pendant is beneficial but not required.

(a) UR3e Teach Pendant. Emergency Stop button in red.

(b) RealSense D435i Stereo RGB-D camera.

Figure 2: Teach Pendant (TP) and RealSense D435i camera.

2.2 Getting the Lab Code

From your host system, open a terminal, `cd` to the `~/Labs` folder, and clone the lab repository:

```
cd ~/Labs
git clone https://github.com/ENEE467-F2025/lab-7.git
cd lab-7/docker
```

Verify that the Docker image has built successfully:

```
docker image ls
```

You should see output similar to:

```
lab-7-hw-image   latest   <container_id>   <time_since_inst>   <size>B
```

Launch the Docker container:

```
docker compose -f lab-7-hw-compose.yml run --rm lab-7-hw-docker
```

You should now see:

```
(lab-7) robot@docker-desktop:~$
```

2.3 Building the Workspace

Inside the container, build and source your ROS 2 workspace:

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
```

2.4 Hardware Setup & Verification

Before starting the hardware procedure, verify that all components of the UR3e system are functional by performing the following steps:

A. **RealSense D435i Stereo RGB-D Camera**:
   - From within a running container, run the following command:

     ```
     realsense-viewer
     ```

     You should see the following window. Confirm that the USB standard (shown on the top-left corner of the app) is at least 3.0 (the lab computers have 3.2).

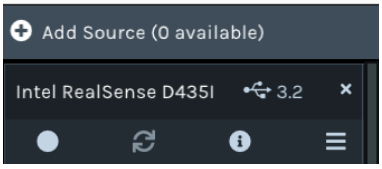


Figure 3: Partial view of the Intel RealSense viewer highlighting the USB standard.

   - Close the RealSense Viewer program by pressing `Ctrl + C` in the previous `tmux` pane. Then run the following command to verify that your camera is configured correctly with ROS 2:

     ```
     ros2 launch realsense2_camera rs_pointcloud_launch.py
     ```

     If you see an RViz window with color, depth, and point cloud streams, then your camera has been setup correctly, and you can terminate the camera node and proceed with the rest of the procedure.

> ⚠ If you see a lower USB standard, close the viewer, **carefully** unplug the RealSense camera (at the camera end and not the PC end), plug it back, and try running the viewer again.

B. **UR3e Robot Arm**:
   - Power on the TP using the button beside the red emergency stop. Once the PolyScope GUI loads, the robot will be in a powered-off state. To enable it, click the bottom-left red button (next to **Power off**), then press ON to switch the robot to the `IDLE` state.
   - Verify that the robot is reachable over Ethernet by running the following command from your container session:

```
ping 192.168.77.22
```

You should see output printed to the terminal indicating successful communication:

```
64 bytes from 192.168.77.22: icmp_seq=49 ttl=64 time=0.329 ms
```

C. **Workspace Props**:
   - Verify that the object (cup) we will use in the pose estimation portion of the procedure is available in the robot's workspace (**Note:** cup colors will vary, but this information is immaterial to the task)

2.5 The Robotiq Hand-E Gripper

The Robotiq Hand-E is a two-finger parallel gripper mounted at the UR3e wrist and used as the end-effector for the grasp phase of the pick-and-place task. The gripper is powered directly by the UR3e tool flange and exposes a single linear degree of freedom: the two fingers move symmetrically between fully open and fully closed. A position command therefore reduces to a single setpoint, with optional speed and force limits.

In this course the gripper is commanded through a pure-Python socket interface (the Robotiq API, reachable on TCP port 63352 through the robot's IP) rather than a `ros2_control` hardware interface. The `robotiq_api_wrapper` package wraps this API in ROS 2: a small driver opens a socket to the gripper, activates it, and exposes move operations. A joint publisher reports the single finger position, and a `joint_state_merger` node combines the gripper joint with the UR3e joints into one `/joint_states` stream. Gripper motion is commanded through an action server that advertises the `gripper_action/GripperAction` action. The goal carries a `desired_position` in the range $[0, 255]$ (0 fully open, 255 fully closed) with optional `desired_speed` and `desired_force`. The PnP routine you will study sends a close goal once the arm reaches the grasp pose and an open goal once it reaches the place pose. You will bring these nodes up below.

2.6 Launching MoveIt 2 for the Arm-Gripper System

Before controlling the robot via ROS 2, we need to start the TP UR3e program. Power ON the TP, if it isn't already powered on, then:

1. Click Power Off → ON at the bottom-left
2. Step out of the robot's workspace, then click the → START button to unlock the robot's brakes and allow it receive programs. You should now see to `Manual` on the bottom left.
3. Tap `Exit` to return to the main GUI. Then tap `Load Program`, and select `ur3e_ros.urp`.
4. Assuming packages in your workspace were built with `--symlink-install`, in a sourced `tmux` session, split panes. In the first pane, start the UR3e ROS 2 hardware driver (`ur_robot_driver`):

   ```
   ros2 launch ur_robot_driver ur_control.launch.py \
   ur_type:=ur3e robot_ip:=192.168.77.22 launch_rviz:=false
   ```

5. Click `Open` then press `Play` to start the program.

The lab computers have been setup to automatically read the configuration file passed as the `kinematics_params_file` , so you do not need to provide those yourself.

> ⚠ **NOTE:** The pendant must remain in `Play` mode for ROS 2 to control the robot. Only click the `Play` button when you are ready to command the robot via ROS 2. The robot must remain in the `IDLE` or `OFF` states at all other times.

2.6.1 Bring Up the Hand-E Gripper Nodes

The Hand-E is driven by a small set of ROS 2 nodes in the `robotiq_api_wrapper` package. With the UR3e driver running, launch them in a new `tmux` pane:

```
ros2 launch robotiq_api_wrapper gripper_jsp.launch.py
```

This launch starts:

- `gripper_joint_publisher`: publishes the single Hand-E finger joint state.
- `joint_state_merger` (from `ur3e_hande_moveit_config`): merges the gripper joint state into the UR3e's states to form a composite robot state on `/joint_states`.
- `gripper_action_server` (from `robotiq_api_wrapper`): connects to the gripper over the socket API (the `robot_ip` parameter defaults to `192.168.77.22`), activates it, and advertises the `robotiq_grip_action` action for asynchronous gripper control.

2.6.2 Plan Motions on the UR3e using MoveIt

MoveIt 2 provides IK, collision checking, trajectory generation, and controller management for the UR3e. Turn ON the TP and start the UR3e ROS 2 driver, if you haven't already. Click `Play` on the TP to ready the robot for programming.

Then, in a new `tmux` pane, launch the MoveIt bringup for the UR3e:

```
ros2 launch ur_moveit_config ur_moveit.launch.py \
ur_type:=ur3e robot_ip:=192.168.77.22
```

RViz will open and load the UR3e's URDF. Using the Move tab on the TP, jog the real robot SLOWLY, and confirm RViz mirrors the motion. If joint states do not update, MoveIt cannot plan safely. Then, maintaining a safe distance from the UR3e, using MoveIt's `MotionPlanning` RViz panel, plan then execute a motion to the `ur_test` configuration by selecting it from the `Goal State` dropdown. You should observe the UR3e move to the selected configuration, with its state evolution mirrored on RViz.

Question 2.

The UR3e ROS 2 driver publishes joint states on the topic: `/joint_states`.

a. Why must MoveIt receive these messages continuously before planning?

b. Terminate the UR3e ROS 2 driver process and try planning to a goal from MoveIt. What

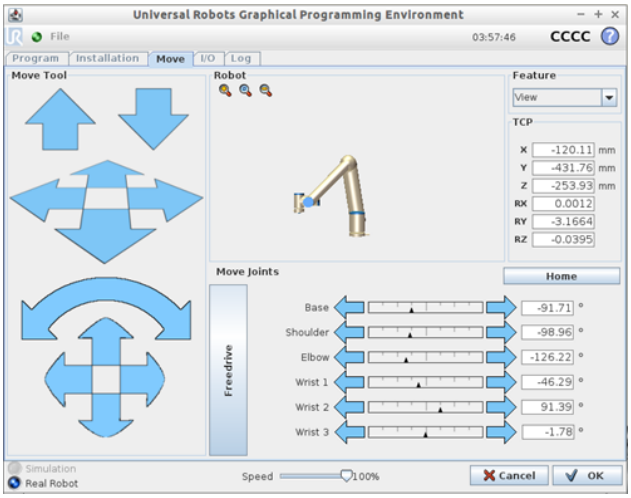


Figure 4: Robot jogging interface on the TP.

failure mode, if any, do you record?

2.6.3 Programmatic Control via PyMoveIT2

Similar to Lab 5, Part 2, we can send programmatic goals to the robot by scripting. Open the `ur3_joint_goal.py` file in the `ur3e_hande_moveit_py` `ament_python` package. Ensure that both the UR3e drivers and MoveIt are active as per the commands introduced earlier. After notifying the team member(s) manning the hardware station to keep a safe distance, the team member(s) sending commands can run:

```
ros2 run ur3e_hande_moveit_py ur3_joint_goal
```

You should observe the robot move to the test configuration earlier. You can home the robot by specifying its named group state within the `group_state` parameter as follows:

```
ros2 run ur3e_hande_moveit_py ur3_joint_goal --ros-args -p \
group_state:="ur_home"
```

2.6.4 Test Hand-E Control

With the gripper nodes running, command the gripper from the command line. After notifying the team member(s) at the hardware station, send a close goal:

```
ros2 action send_goal /robotiq_grip_action gripper_action/action/GripperAction \
"{desired_position: 255, desired_speed: 10, desired_force: 10}"
```

and then an open goal:

```
ros2 action send_goal /robotiq_grip_action gripper_action/action/GripperAction \
"{desired_position: 0, desired_speed: 10, desired_force: 10}"
```

Question 3. What are the goal, result, and feedback fields of the `gripper_action/GripperAction` action? How does the single `desired_position` value relate to the linear, one-DOF nature of the Hand-E?

3. Point Cloud-Based Object Pose Estimation

3.1 Test the Perception Nodes

The RealSense D435i driver (`realsense2_camera`) publishes several visual message streams:

- `/camera/camera/color/image_raw`: RGB image
- `/camera/camera/depth/image_rect_raw`: registered depth
- `/camera/camera/color/camera_info`: intrinsic calibration
- `/camera/camera/depth/color/points`: aligned organized point cloud

Terminate all running ROS 2 processes. Then bring up the camera and point cloud perception nodes using the following command (ensure the cup is in the camera's field-of-view):

```
ros2 launch ur3e_hande_perception perception.launch.py
```

The pipeline detects an object by clustering point clouds and fitting clusters to a prespecified known geometry within bounds. The command above starts nodes that perform:

- Voxel filtering: reducing point cloud density
- Cropping: removing irrelevant workspace regions
- Plane segmentation: extracting tabletop surfaces
- Object clustering: grouping and bounding-boxing objects
- 6D pose extraction: estimating object centroids and rough extents

An RViz window will pop-up displaying an RGB image overlay, a filtered cloud, plane markers (green), and an object bounding box shown in cyan here, though object colors may vary (see Figure 5).

If the perception node detects multiple objects in the workspace or no object at all, you will need to adjust the `crop_bounds` parameter used by the `pc_voxel_filter_node`. This parameter can be found in the `perception.launch.py` file located in the `launch` directory of the `ur3e_hande_perception` `ament_python` package. Modify the x and y bounds (either increasing or decreasing them based on your situation), then re-run `perception.launch.py`.

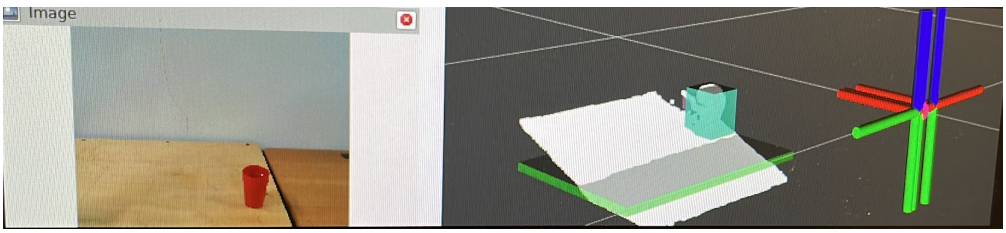


Figure 5: RViz window at camera-only bringup showing camera frames and RViz box markers highlighting the detected object (cup) and segmented plane (table).

We can check the pose of the detected object that provides the input for grasping and place pose computation by listening to the following topic:

```
ros2 topic echo --once /object_detected
```

If an object matching the provided dimensions is found within the camera's view, you should see the action server return its parameters should see its parameters:

```
object_id: 0
position:
x: -
y: -
z: -
dimensions: height: <> width: <> thickness: <>
```

The objects are detected by forming clusters from a filtered point cloud represented by the `PointCloud2` ROS 2 message type (see PointCloud2 Message) on the topic `/filtered_cloud`.

Question 4.

Try echoing a single stream from the `/filtered_cloud` topic. (Hint: use:

```
ros2 topic echo --once /filtered_cloud
```

(a) What camera stream does the value of the `frame_id` in the `PointCloud2` message above relate to?

(b) How does this frame relate to pose estimation? (**Hint**): see From Meters to Pixels.

3.2 Querying for Poses via a Custom Action

While manually querying for poses via the CLI is snappy, it will not afford other ROS 2 nodes access to this information. A simple way of providing such access is via an action server. Start the perception nodes if they aren't running:

```
ros2 launch ur3e_hande_perception perception.launch.py
```

Then, in another `tmux` pane, manually request an *eye-balled* object position (replace the target position values below with the values from the yellow box above as well as values outside the robot's workspace, like $x \geq 1.25$, $y \geq 1.0$, and $z \geq 1.5$, and observe the result returned by the action server:

```
ros2 action send_goal /get_target_obj_pose \
ur3e_hande_planning_interfaces/action/GetTargetObjPose \
"{target_position: {x: 0.28, y: 0.50, z: 0.05}, \
target_height: 0.12, target_obj_bounds: [0.20, 0.33, 0.0, 0.0]}"
```

The parameter `target_obj_bounds` contains the approximate location of the XY-coordinate of the object on the UR3e's mounting table. You should see the object's position printed to the terminal in the former case as well as the action server reporting success, confirming that an object satisfying the above description exists within the camera's field of view.

Question 5.

a. Which ROS 2 message types define the Goal, Result, and Feedback of the above action? **Hint**: Run

```
ros2 interface show ur3e_hande_planning_interfaces/action/GetTargetObjPose
```

b. Why is an action the preferred choice in this setting over say a service? In particular, what could go wrong in a PnP task if our robot's control node relied on a ROS 2 service for object poses?

Exercise 2. Simple Pick-and-Place Action Client (35 Pts.)

In this exercise, you will complete and extend the autonomous pick-and-place routine implemented in `pnp_demo.py`, located within the `ur3e_hande_moveit_py` `ament_python` package. This node serves as a ROS 2 action client that integrates motion planning, inverse kinematics, and gripper control into a full pick-and-place behavior. You verified the inverse-kinematics approach in simulation (Exercise 1). Here you apply the same approach to the hardware node and run it on the real UR3e. Currently, this code assumes that IK always succeeds, the gripper is always available, and placement positions are always valid. Obviously this is not always the case and your task will be to fix these assumptions and extend the PnP behavior.

Deliverables:

IK Computation for Each Sub-Task:

a. Modify the `TODO` highlighted block to get the IK result for each sub-task using the `solve_ik` function:

```
self.approach_q = self.solve_ik(self.approach_pos, seed=home_q)
```

The task progression is as follows (gripper actions in blue):

Home → Approach → Pre-Grip → Grasp → Lift → Place → Release → Home

IK Failure Handling:

b. The method `solve_ik()` may return `None`. Modify the code so that if any of `approach_q`, `pre_grip_q`, or `place_q` are `None`, the node logs an error and aborts the sequence safely.

Synchronous Execution:

c. Setting the launch parameter `synchronous` equal to `True` ensures that the motion for each sub-task completes before the next begins (i.e., waits on `wait_until_executed()` after every `timed_motion()` call). What happens when `synchronous` is set to `False`?

Files to Submit:

Submit the following:

- A code snippet showing your added IK failure handling.
- A code snippet showing your IK logic

Note: After completing your implementation, you will be able to test your PnP routine on the real UR3e with the Hand-E gripper, time permitting. Your grade for this exercise will be based on your submitted Python code and your answer to part (c).

4. Conclusion and Lab Report Instructions

After completing the lab, summarize the procedure and results into a well written lab report. Include your solutions to all of the question boxes in the lab report. Refer to the List of Questions to ensure you don't miss any. Include your full solution to the exercises in your lab report. If you wrote any Python programs during the exercises, include these files as a separate attachment, appending your team's number and last names (in alphabetical order) to the file, e.g., **Jane Doe**, **Alice Smith**, and **Richard Roe** in **Team 90** submitting their modified `main.py` would submit the file `main_Team90_Doe_Roe_Smith.py` and the report `Lab_<LabNumber>_Report_Team90_Doe_Roe_Smith.pdf`, where `<LabNumber>` is the number for that week's lab, e.g., **0**, **1**, **2**, etc. Submit your lab report along with any code to ELMS under the assignment for that week's lab. You must complete and submit your lab report by **Friday, 11:59 PM** of the week following the lab session. See the **Files** section on ELMS for a lab report template named `Lab_Report_Template.pdf`. The grading rubric is as follows:

Component	Points
Lab Report and Formatting	10
Question 1.	5
Question 2.	5
Question 3.	5
Question 4.	5
Exercise 1.	35
Exercise 2.	35
Total	100